Quest 14: The Shardorithm

Lord of the Wings · binding the pieces of a shattered window to its frame

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION
A window is a grid of glass pieces, one symbol per piece. Bindings go along pixel edges between two pieces, or between a piece and the frame. Count the bindings needed to hold the window together.

INPUT
A rectangular grid of symbols. Each symbol is one piece: a single group of pixels joined up, down, left or right.

OUTPUT
One number: how many bindings to make.

WHAT CHANGES BETWEEN PARTS
Part I: bind every edge. Part II: every piece only has to reach the frame, but each join binds a whole seam; find the cheapest set of joins. Part III: like Part II, with two free charges that each bind every seam of one piece.

The story

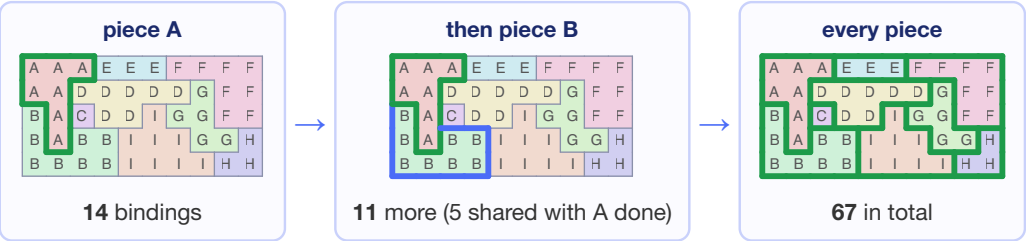
The portals now ask *where* to go instead of *when*, and each one shows its destination through a window whose glass is shattered. A portal only opens once its glass is bound back together and to its frame, so the narrator mends three windows (home, Nulldor, and finally the sulphur mines where the team waits) while spending as little magic as possible.

Understanding the question

The rules

- A **binding** covers one pixel edge. It goes where two **different** pieces touch, or where a piece touches the **frame** (the outside of the grid). Edges inside one piece never need one.
- Part I: bind **all** such edges. Each shared edge counts once, not once per piece.
- Parts II and III: a **seam** is every edge two pieces share (or a piece shares with the frame). You either bind a whole seam or none of it, and every piece must end up connected to the frame, directly or through other pieces.

The Part I example



Thick lines are bindings; in the middle panel A's are green, B's new ones blue. Each piece has its own pale colour.

Extra examples of my own (run through copies of my solutions)

Window	`AB`	`AAB`
		`CCB`
			`ABBBBC`	`AAA`
				`ABA`
				`AAA`

Part I	7	14	16	16
Part II	4	5	5	16
Part III	0	0	0	0
Why (Part II)	join A–B (1), attach one side (3)	A–B, C–B (1 each), then A's frame seam (3)	the ends join B (1 each), B's frame seam is 8, the ends' are 3: use A's	B only touches A, so nothing to choose: 12 + 4

With two charges every one of these is free: there are at most two pieces to charge, or charging B binds it to everything.

Shared approach: the window as a graph of seams

Parts II and III first turn the grid into a **graph**: each piece is a point, plus one extra point for the frame, and each seam is a link whose **cost** is its length in pixel edges. Walking every pixel once and comparing it with the pixel to its left and above (and with the frame on the border) counts every seam. The small window has 24 links:

> **1:** `A–E` `E–F` `F–D` `A–C` `C–B` `D–B` `F–H` `I–H`
> **2:** `A–D` `D–G` `D–C` `B–I` `G–H`
> **3:** `frame–E` `E–D` `D–I`
> **4:** `frame–H` `frame–I` `F–G` `I–G`
> **5:** `frame–A` `A–B`
> **7:** `frame–F` `frame–B`

Links grouped by cost (seam length), from an instrumented copy of my Part II solution. Adding them all up gives 67, Part I's answer.

PART I Bind every edge

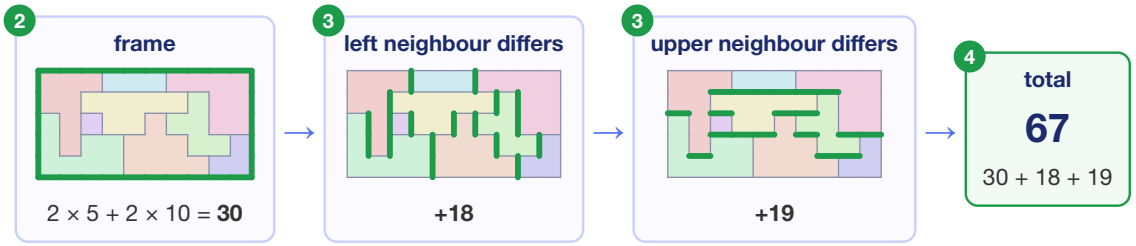
How many bindings are needed if every edge between two different pieces, and every edge on the frame, gets one?

Step-by-step solution

1. Read the window as a grid of symbols.
2. Bind the frame: one binding per border edge, 2 × rows + 2 × columns.
3. Go through every pixel. Add 1 if the pixel to its left is a different piece, and 1 if the pixel above is.
4. Print the total.

How my solution works

No graph needed: the frame always needs one binding per border edge, so start from 2 × rows + 2 × columns. Then look at every pixel and compare it only with its **left** and **upper** neighbour. Every inside edge has exactly one pixel to its right or below, so each is counted exactly once.



Dots match the step numbers above. Counts from an instrumented copy of my Part I solution on `sample1.txt`. On the bigger window (Part II's second example) it gives 80 for the frame and 279 in total.

PART II The cheapest way to hold together

Every piece must reach the frame through bound seams, and each seam is all or nothing. What is the fewest bindings?

The idea: a minimum spanning tree

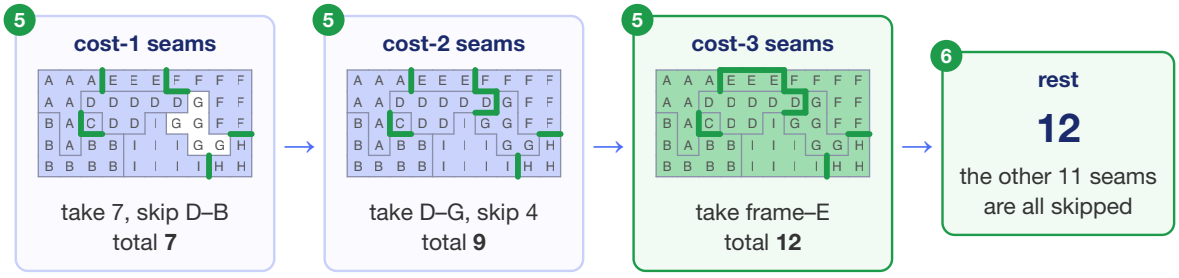
The chosen seams must join all points, and a closed loop of seams is wasted (drop one and all stays joined). So the answer is the cheapest **tree** over all points: a **minimum spanning tree**, built here with **Kruskal's algorithm**: try seams cheapest first and keep one only if it joins two groups not yet connected.

Step-by-step solution

1. Read the grid and treat the frame as one extra piece.
2. Measure every seam: each border pixel adds 1 to its piece's frame seam; each pair of different left/upper neighbours adds 1 to their seam.
3. Sort the seams from cheapest to dearest.
4. Give every piece, and the frame, its own group name.
5. Take the next seam. If both ends are already in the same group, skip it; otherwise pay its cost and merge the two groups. ↻ repeat
6. Print the total paid.

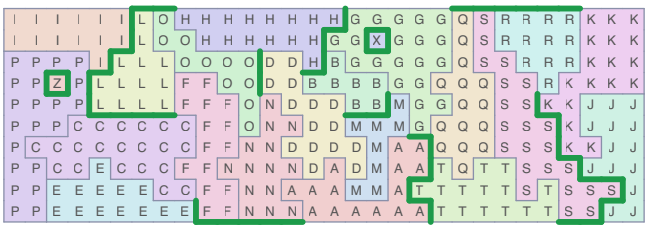
How my solution works

New since Part I: count each seam's length separately (the graph above), then pick seams instead of adding everything. Each piece carries a group name; joining two groups renames every member of one to the other's name.



Dots match the steps above; from an instrumented copy. Green lines: seams taken so far. Cells are coloured by group (white = still alone, blue = a group not yet on the frame, green = joined to the frame). D–B is skipped because C–B and A–C already linked them.

Worked examples



The big window: 22 seams, **63** bindings; the small one above: 9 seams, **12**. Both match the puzzle. A tree over *n* pieces plus the frame always has exactly *n* seams. Several groups reach the frame through a single piece, like L, O and Q along the top.

PART III Two free charges

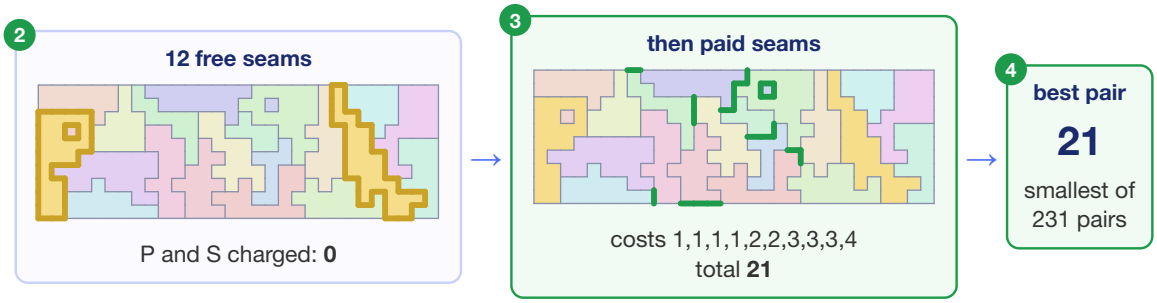
Two charges each bind every seam of one chosen piece for free. Which two pieces make the remaining repair cheapest?

Step-by-step solution

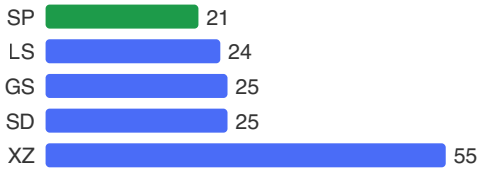
1. Measure every seam, as in Part II.
2. Pick the next pair of pieces to charge: every seam touching either one now costs 0.
3. Sort and merge groups exactly as in Part II, adding up what is paid.
4. After trying every pair of pieces this way (steps 2 and 3 for each), print the smallest total.

How my solution works

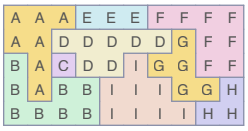
New since Part II: charging a piece makes all its seams, including its frame seam, cost **0**. Kruskal takes those free seams first, and then finishes as before. Charging can only lower costs, so using both charges is never worse. My solution simply **tries every pair** of pieces (36 pairs for the small window, 231 for the big one) and keeps the cheapest tree.



Big window, from an instrumented copy. Gold cells: the charged pieces P and S; gold lines: seams the charges bind (the frame seams of P and S plus their 10 neighbours); green: the 10 seams Kruskal still pays for. Dots match the steps above.



Big window: the 4 best pairs and the worst (XZ), additional bindings.



Small window: charging A and G binds everything, **0**. Five other pairs (AH, FD, FB, DI, DH) also give 0.

Edge case: a one-piece window has no pairs to try, so the answer stays "infinity" (a copy run on `AAA/AAA` prints `Infinity`, not 0). **Fragile (Parts II and III):** a seam is found again "the other way round" by reversing its two-name label's text, which only works because every piece is one character and the frame's name reads the same backwards; a space as a piece symbol would break it. The real inputs use punctuation such as `#` but no spaces, so it holds. Each pixel edge also scans the whole seam list, which is slow on big windows, though Part III still runs in under a second.

Summary

Part	What gets bound	Method	Small window	Big window
I	every edge between pieces and on the frame	count frame edges + pixels whose left / upper neighbour differs	67	279
II	whole seams, just enough to join every piece to the frame	graph of seam lengths → minimum spanning tree (Kruskal)	12	63
III	as II, but two pieces' seams are free	try every pair of pieces: zero their seams, then Kruskal	0	21

Key ideas: count each shared edge once by looking only left and up. Turn the grid into a **graph** (pieces plus the frame as points, seam lengths as costs), and "hold together as cheaply as possible" becomes a **minimum spanning tree**. Free charges are just zero-cost links, and with few pieces, **trying every pair** is cheap enough.